Security Engineering - Lab assignment
MessageBoard using νActionGUI

In this lab assignment you will implement another application using νActionGUI. We will only focus on making the application secure.
Appendix 2 summarizes the important parts of the νActionGUI workflow.

1 Message Board running example

Consider a simple application where people can post and reply to messages.

1.1 Data model

A `Person` can post a `Message` that consists of a `title` and a `text`. Additionally, a `Person` can post a `Reply` to an existing `Message`. A `Reply` contains only `text`. A `Person` has only a `name`.

Message Board interface consists of the following web pages:

- **Main page** Shows the messages and allows their management.
- **Create Message page** Allows the creation of a new message.
- **Edit Message page** Used to modify an already existing message.
- **View Message page** Used to see the messages' content and replies.
- **Create Reply page** Create a new reply to a given message.
- **Edit Reply page** Can be used to modify a reply.
- **View Reply page** Used to see the detailed content of a reply.

1.2 Security model

The application includes the following roles: Unauthenticated role is named `VISITOR`, authenticated roles can be `NORMAL` (default) or `ADMIN`.

Download the `messageBoardInit.zip` which contains the code similar to the one described in this lab. In this initial version, the data model contains enough information for the first task in Section 1.2, the security model allows every action (i.e., every role has `fullAccess` permission to every entity), and the privacy model does not define any personal data (i.e., it is empty).

Part 1: Enforcing security policies

Enforce the following security policies (the main task of this part):

`VISITOR` Unauthenticated users can read the names of other users. They can also read the `title`, `text`, and `messageReplies` properties of `Message` as well as the `text` attribute of `Reply`.

`NORMAL` User with `NORMAL` role can do whatever unauthenticated users can do. Furthermore, they can:

1. Create messages.
2. Delete their own message, unless the message has replies.
3. Read the `messageOwner` of their own messages.
4. Update the `title` and `text` of their own messages.
5. Initialize the `messageOwner` to themselves, or update it to `null` for their own messages.
6. Create replies.
7. Delete their own reply.
8. Initialize the `replyOwner` to themselves.
9. Update the `text` of their own replies.
10. Read the `replyOwner`.
11. Add their own reply to the `messageReplies` of a message, unless it is already added.
12. Remove their own reply from the `messageReplies` of a message, if it is present.

`ADMIN` Users with `ADMIN` role can do what `NORMAL` users can do.

Part 2: Adding a new role

In this part, we extend the application with a new role `MODERATOR` with additional rights. Particularly, we

1. Create a new **Update User Window** that allows users to update their name and role.

   (a) Extend the data model with the new entrypoints for displaying the **Update User Window** and updating the user per se. Add the following methods to the `Person` entity:

```
1     @entry updateUserWindow()
2     @entry updateUser()
```

   (b) Implement the functionality of these entrypoints in the implementation (i.e., `project.py` and the HTML templates).

   - Copy the following code and put it at the end of `project.py`.
   - Create `UpdateUserWindow.html` in the `templates` folder, and paste this code into it.
   - Add the following HTML button to the `main.html` on the line 33:

```
1     <button class="btn btn-info" onclick="
          updateUser()">
2         Update User Info
3     </button>
```

     and inside the `<script>` tag:

```
1     function updateUser() {
2       uid = {% if user.id %}
3       {{ user.id }}
4       {% else %}
5       null
6       {% endif %};
7       window.location.href='/updateUserWindow?id=
            ${uid}';
8     }
```

     The resulting new `main.html` can be copied from here.

2. Update the security policy (the main task of this part):

- `NORMAL` users can now read their own `role`, and update their own `name` and `role`. For names, the updated value cannot be an empty string. For roles, the updated value cannot be the unauthenticated role (we can safely assume that the value is of type Role).
- Add a new `MODERATOR` role.
- Update the security model: `MODERATOR` users can do whatever `NORMAL` users can. Additionally, they can delete and update the `title` and `text` of any message, and update `text` of any reply (regardless of the author or of the presence of replies to a message).

Part 3: Adding a new functionality

Extend the application with a new functionality:

> A Person can create private messages that are shared with a group of friends and only Persons in this group can view and reply to the message.[1] A message is considered public if it is not shared with anyone in particular.

This can be done in three steps:

1. Extend the data model with the new many-to-many association

   $$\texttt{Privatemessages} \subseteq \texttt{Person} \times \texttt{Message}$$

   with the association end `friendMessages` in `Person` and `sharedWith` in `Messages`. The additional lines needed are respectively:

```
entity Person {
    //...
    Set(Message) friendsMessages in Privatemessages
}
entity Message {
    //...
    Set(Person) sharedWith in Privatemessages
}
```

[1] To view a message means to read the `Message` entity.

2. Modify the security model to account for the new functionality (the main task of this part). What are reasonable permissions for `sharedWith` and `friendsMessages`? What about permissions for replying to private messages?

3. Modify the `CreateMessageWindow` and `EditMessageWindow` windows by adding a list of all users and allowing the user to choose multiple other users with whom to share the message. Modify the function implementation in `project.py`.

2 Development Workflow: Checklist

To start for the first time make sure you have Docker installed. You can download the latest version of Docker from here.

In order to use νActionGUI, download the `NuActionGUI` file from the course's Moodle page and extract it. The `docker-compose.yml` file contains a Linux instance with python standard libraries installed.

To start for the first time, navigate to the directory that contains the `docker-compose.yml` file, and type:

```
$ docker compose up --build
```

This will build the images and run the container (named `nag-app`) for the first time. Once the image has been built, you can omit `-build` flag when starting.

Then, to open a shell to the `nag-app` container:

```
$ docker exec -it nag-app bash
```

Here we assume you have installed νActionGUI using Docker, started the containers, and have a shell in the `nag-app` container.

Since the folder `models/` in the container and in the local are synced, you can edit your models locally using your favorite code editor and see these changes reflected on the container.

For a new project, create a sub-directory in the folder `models/`. In this sub-directory, you can define your models (data, security, and privacy models).

Generating the project Once you have the models defined, go to the `src/` folder and build a virtual environment in python and install necessary packages inside this environment:

```
$ python3 -m venv .venv
$ . .venv/bin/activate
(.venv) $ pip3 install -r ../requirements.txt
```

Then, to generate the project for the first time, in the virtual environment:

```
(.venv) $ python3 generate.py -p <project_name> -o <
   output_folder>
```

Replace `<project_name>` with the desired project name from the `models/` folder. This will create a new folder `<project_name>` in the output folder `<output_folder>/`. The `-o` option is optional, by default, the output folder will be `output`. You may need to adjust the permissions of the bash script in the container (e.g., `chmod -R +x run.sh`) in order to execute it on the host machine.

Regenerate part of the project For now, we assume the data model is fixed and will not be changed during the tutorial. If the security model and/or the privacy model in the `models/<project_name>` changes, to regenerate only the related artifacts, go to the `src/` folder:

```
(.venv) $ python3 generate.py -p <project_name> -re
```

Running the project To execute the compiled web application, go to the `/<output_folder>/<project_name>` folder:

```
(.venv) $ flask --app app.py run --host=0.0.0.0
```

The app should be reachable from your local machine at `localhost:5000/`.